Final project Data Science

Jubilee Mwasa

2026-10-24

```
library(ggplot2)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyr)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.1     v readr     2.2.0
## v lubridate 1.9.5     v stringr   1.6.0
## v purrr     1.2.2     v tibble    3.3.1

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(pheatmap)

gene_expression <- read.csv("~/Downloads/GSE85307_geneExpression.csv")
gene_names<- read.csv("~/Downloads/GSE85307_geneNames.csv")
participant_information<-read.csv("~/Downloads/GSE85307_covar-1.csv")


# Check if sample IDs match
sum(colnames(gene_expression)[-1] %in% participant_information$geoID)
```

```
## [1] 157
```

```r
# Check if probe IDs match gene names
sum(gene_expression$X %in% gene_names$PROBEID)
```

```
## [1] 500
```

```r
#  only samples that have matching participant information
matched_ids <- intersect(
  colnames(gene_expression)[-1],
  participant_information$geoID
)

# Select those samples and keep the probe ID column
expression_matched <- gene_expression[
  , c("X", matched_ids)
]

# Keep participant records in the same order as the expression samples
participant_matched <- participant_information[
  match(matched_ids, participant_information$geoID),
]

# Confirm the sample IDs are now in the same order
all(colnames(expression_matched)[-1] == participant_matched$geoID)
```

```
## [1] TRUE
```

```r
# Add the gene names to the expression data using probe IDs
gene_expression_named <- merge(
  gene_names,
  expression_matched,
  by.x = "PROBEID",
  by.y = "X"
)

# Check the result
head(gene_expression_named[, 1:3])
```

```
##   PROBEID    GENE GSM2264363
## 1 7897322   PHF13   7.197650
## 2 7897860 TNFRSF8   8.094866
## 3 7898167    CTRC   6.504210
## 4 7898653  FAM43B   9.015528
## 5 7898809   EPHB2   7.022149
## 6 7900468    NFYC   9.908033
```

```r
#new function (Step 4) will make all the plots, so the IL24-only code isn't needed anymore.
#il24_row <- gene_expression_named[
#  gene_expression_named$GENE == "IL24",
#]

#il24_row
```

```
# Select IL24 gene expression
#il24_data <- gene_expression_named %>%
#  filter(GENE == "IL24")

# Convert gene expression data to long format
#il24_long <- il24_data %>%
#  select(-PROBEID, -GENE) %>%
#  pivot_longer(
#    cols = everything(),
#    names_to = "geoID",
#    values_to = "IL24_expression"
  #)

# Match IL24 expression to participant information
#il24_analysis <- il24_long %>%
#  left_join(participant_matched, by = "geoID")

# Check the data
#head(il24_analysis)
#nrow(il24_analysis)


#plot1 <- ggplot(il24_analysis, aes(x = IL24_expression)) +
#  geom_histogram(
#    bins = 20,
#    fill = "#8AB9B0",
 #   color = "white",
 #   linewidth = 0.4,
 #   na.rm = TRUE
 # ) +
 # labs(
 #   title = "Distribution of IL24 Gene Expression",
 #   x = "IL24 Gene Expression",
 #   y = "Number of Participants"
#  ) +
#  theme_classic(base_size = 14) +
#  theme(
#    plot.title = element_text(
#      face = "bold",
#      hjust = 0.5,
#      size = 16
 #   ),
 #   axis.title = element_text(face = "bold", size = 13),
 #   axis.text = element_text(size = 11, color = "black")
#  )

#print(plot1)



# PLOT 2: SCATTERPLOT
# IL24 Gene Expression vs. Maternal Age
# Convert maternal age to numeric
```

```
#il24_analysis$mat_age_yrs <- as.numeric(
#  sub(".*: ", "", as.character(il24_analysis$mat_age_yrs))
#)

# Check the result
#str(il24_analysis$mat_age_yrs)
#summary(il24_analysis$mat_age_yrs)

#plot2 <- ggplot(
#  il24_analysis,
#  aes(x = mat_age_yrs, y = IL24_expression)
#) +
 # geom_point(
 #   size = 2.2,
  #  alpha = 0.65,
  #  color = "#C06C84",
#    na.rm = TRUE
#  ) +
 # geom_smooth(
 #   method = "loess",
 #   se = TRUE,
 #   color = "#765285",
 #   fill = "#D9C2E9",
 #   na.rm = TRUE
 # ) +
 # labs(
 #   title = "IL24 Gene Expression by Maternal Age",
  #  x = "Maternal Age (years)",
  #  y = "IL24 Gene Expression"
 # ) +
 # theme_classic(base_size = 14) +
 # theme(
#    plot.title = element_text(
 #     face = "bold",
 #     hjust = 0.5,
  #    size = 16
  #  ),
  #  axis.title = element_text(face = "bold", size = 13),
   # axis.text = element_text(size = 11, color = "black")
  #)

#print(plot2)




# PLOT 3: BOXPLOT
# IL24 Expression by Preeclampsia Status and BMI

# Extract the numeric BMI values from the original text
# Make the grouping variables easier to interpret
# PLOT 3: IL24 Expression by Preeclampsia Status and BMI
```

```
# Step 1: Convert BMI to numeric and create BMI categories
#il24_analysis <- il24_analysis %>%
#  mutate(
 #   mat_BMI = as.numeric(
 #     sub(".*: ", "", as.character(mat_BMI))
 #   ),
 #   BMI_category = cut(
 #     mat_BMI,
 #     breaks = c(-Inf, 18.5, 25, 30, Inf),
 #     labels = c(
  #      "Underweight",
  #      "Normal weight",
  #      "Overweight",
   #     "Obesity"
   #   ),
   #   right = FALSE
   # )
  #)

# Step 2: Set category order
#il24_analysis <- il24_analysis %>%
#  mutate(
#    preg_preeclampsia = factor(preg_preeclampsia),
#    BMI_category = factor(
#      BMI_category,
 #     levels = c(
 #       "Underweight",
  #      "Normal weight",
   #     "Overweight",
   #     "Obesity"
   #   )
  #  )
#  )

# Step 3: Check sample sizes
#table(
#  il24_analysis$preg_preeclampsia,
#  il24_analysis$BMI_category,
#  useNA = "ifany"
#)

# Step 4: Create boxplot with jittered individual observations
#plot3 <- ggplot(
#  il24_analysis,
#  aes(
#    x = preg_preeclampsia,
#    y = IL24_expression,
 #   fill = BMI_category
#  )
#) +
#  geom_boxplot(
#    position = position_dodge(width = 0.8),
 #   na.rm = TRUE,
```

```
#   color = "#51445F",
#   linewidth = 0.4,
#   alpha = 0.75,
 #  outlier.shape = NA
# ) +
# geom_point(
#   aes(color = BMI_category),
#   position = position_jitterdodge(
    # jitter.width = 0.15,
  #   dodge.width = 0.8
  # ),
  # alpha = 0.55,
  # size = 1.5,
  # na.rm = TRUE
# ) +
# scale_fill_manual(
#   values = c(
#     "Underweight" = "#B8A1D9",
#     "Normal weight" = "#E8A0BF",
 #    "Overweight" = "#E9B872",
 #    "Obesity" = "#8AB9B0"
#   ),
 #  drop = FALSE
# ) +
# scale_color_manual(
#   values = c(
#     "Underweight" = "#8065A8",
 #    "Normal weight" = "#B85F85",
 #    "Overweight" = "#B27A32",
#     "Obesity" = "#4D8278"
 #  ),
  # guide = "none",
  # drop = FALSE
# ) +
# labs(
#   title = "IL24 Expression by Preeclampsia Status and BMI",
#   x = "Pregnancy Outcome",
#   y = "IL24 Gene Expression",
#   fill = "Maternal BMI Category"
# ) +
# theme_classic(base_size = 14) +
# theme(
  # plot.title = element_text(
  #   face = "bold",
  #   hjust = 0.5,
  #   size = 16
  # ),
 #  axis.title = element_text(face = "bold", size = 13),
 #  axis.text.x = element_text(
  #   size = 11,
  #   color = "black"
  # ),
  # axis.text.y = element_text(
```

```
   #   size = 11,
  #    color = "black"
  #  ),
  #  legend.position = "right",
 #   legend.title = element_text(face = "bold"),
 #   plot.margin = margin(10, 15, 10, 10)
#  )

# Step 5: Display plot
#print(plot3)
```

```
#selected_genes <- c("ABCA9", "ABCB7")



#participant_plot_data <- participant_matched %>%
 # mutate(
  #  mat_age_yrs = as.numeric(
  #    sub(".*: ", "", as.character(mat_age_yrs))
  #  ),
  #  mat_BMI = as.numeric(
   #   sub(".*: ", "", as.character(mat_BMI))
  #  ),
  #  preg_preeclampsia = as.factor(preg_preeclampsia),
  #  BMI_category = case_when(
 #     is.na(mat_BMI) ~ NA_character_,
 #     mat_BMI < 18.5 ~ "Underweight",
 #     mat_BMI < 25 ~ "Normal weight",
 #     mat_BMI < 30 ~ "Overweight",
  #    mat_BMI >= 30 ~ "Obese"
  #  )
 # )

# Generate three plots for each selected gene
#for (gene_name in selected_genes) {

  # Extract expression data and attach participant information
  #gene_data <- gene_expression_named %>%
  #  select(-PROBEID, -GENE) %>%
  #  pivot_longer(
   #   cols = everything(),
    #  names_to = "geoID",
    #  values_to = "expression"
   # ) %>%
   # mutate(expression = as.numeric(as.character(expression))) %>%
   # left_join(participant_plot_data, by = "geoID")

  # Stop with a clear message if the gene is absent
 # if (nrow(gene_data) == 0) {
  #  message("Gene ", gene_name, " was not found in gene_expression_named.")
   # next
 # }
```

```
 # 1. Histogram: distribution of gene expression
# p1 <- ggplot(gene_data, aes(x = expression)) +
 #  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
 #  labs(
 #    title = paste("Distribution of", gene_name, "Expression"),
 #    x = "Gene Expression",
 #    y = "Frequency"
 #  ) +
 #  theme_minimal()

# print(p1)

 # 2. Scatterplot: gene expression vs maternal age
# p2 <- ggplot(
  # gene_data,
  # aes(x = mat_age_yrs, y = expression)
# ) +
 #  geom_point(na.rm = TRUE) +
#   geom_smooth(method = "lm", se = TRUE, na.rm = TRUE) +
 #  labs(
 #    title = paste(gene_name, "Expression vs Maternal Age"),
  #   x = "Maternal Age (years)",
  #   y = "Gene Expression"
 #  ) +
 #  theme_minimal()

# print(p2)

 # 3. Boxplot: gene expression by preeclampsia status and BMI category
# p3 <- ggplot(
 #  gene_data,
 #  aes(
#     x = preg_preeclampsia,
 #    y = expression,
 #    fill = BMI_category
 #  )
 #) +
 #  geom_boxplot(na.rm = TRUE) +
 #  labs(
 #    title = paste(gene_name, "Expression by Preeclampsia and BMI"),
 #    x = "Preeclampsia Status",
 #    y = "Gene Expression",
 #    fill = "BMI Category"
 #  ) +
 #  theme_minimal()

# print(p3)
#}


#Final submission code -CLEAN THE PARTICIPANT (COVARIATE) DATA
# In the raw file, each value is stored as text with the variable
# name in front, e.g. "mat_age_yrs: 31". R can't do math on that,
```

```r
# so we remove the label and convert each column to the right type.


# Part A: remove the "name: " label from every column at once
participant_clean <- participant_matched %>%   # start from the matched participants
  mutate(                                      # mutate() changes or creates columns
    across(                                    # across() applies the same change to many columns
      -c(subjectID, geoID),                    # every column EXCEPT the two ID columns
      ~ trimws(                                # trimws() removes extra spaces at the start/end
          sub(".*: ", "",                      # sub() finds "anything followed by ': '" and replaces i
              as.character(.x))                # .x is the current column, turned into text first
        )
    )
  )
# Example: "mat_age_yrs: 31" becomes "31"
# (Still text at this point, not a number yet)

# Part B: give each column the right type
participant_clean <- participant_clean %>%
  mutate(
    # Continuous variables: turn the text "31" into the number 31
    across(c(mat_age_yrs,              # maternal age (years)
             mat_BMI,                  # maternal BMI (kg/m^2)
             mat_vitD,                 # maternal vitamin D (ng/mL)
             delivery_gestAge_wks),    # gestational age at delivery (weeks)
           as.numeric),

    # Categorical variable 1: pregnancy outcome
    # toTitleCase() capitalizes the first letter (e.g. "normal" -> "Normal")
    # so the plot labels look tidy; factor() tells R it is a category
    preg_preeclampsia = factor(tools::toTitleCase(preg_preeclampsia)),

    # Categorical variable 2: BMI category, made from continuous BMI
    # (the assignment requires one categorical variable derived
    # from a continuous one; this is it)
    BMI_category = cut(
      mat_BMI,                                  # the number we are grouping
      breaks = c(-Inf, 18.5, 25, 30, Inf),      # WHO cut-points
      labels = c("Underweight",                 # below 18.5
                 "Normal weight",               # 18.5 to under 25
                 "Overweight",                  # 25 to under 30
                 "Obesity"),                    # 30 and above
      right = FALSE                             # left edge included: BMI of exactly 25 counts as Over
    )
  )

#  check that the cleaning worked

# How many participants are in each outcome group?
# (should show exactly 2 groups; useNA shows any missing values)
table(participant_clean$preg_preeclampsia, useNA = "ifany")
```

```
##
```

```
##       Normal Preeclampsia
##          110           47
```

```r
# How many participants are in each BMI category?
table(participant_clean$BMI_category, useNA = "ifany")
```

```
##
##   Underweight Normal weight    Overweight       Obesity
##             5            48            47            57
```

```r
# Min, mean, max etc. for each continuous variable,
# to check the numbers look realistic (e.g. ages roughly 18-45)
summary(participant_clean[, c("mat_age_yrs", "mat_BMI", "mat_vitD",
                              "delivery_gestAge_wks")])
```

```
##   mat_age_yrs       mat_BMI         mat_vitD     delivery_gestAge_wks
##  Min.   :18.43   Min.   :17.95   Min.   : 5.11   Min.   :26.43
##  1st Qu.:22.15   1st Qu.:22.90   1st Qu.:13.40   1st Qu.:38.14
##  Median :26.57   Median :27.33   Median :20.40   Median :39.29
##  Mean   :26.38   Mean   :28.94   Mean   :22.98   Mean   :38.82
##  3rd Qu.:30.14   3rd Qu.:33.27   3rd Qu.:29.50   3rd Qu.:40.00
##  Max.   :38.76   Max.   :61.95   Max.   :80.80   Max.   :42.00
```

```r
# COMBINE GENE EXPRESSION + PARTICIPANT INFO
# Goal: one table where each ROW is a participant and there is
# one COLUMN per gene (IL24, ABCA9, ABCB7), next to their
# age, BMI, outcome etc. This is the format ggplot needs.

# The three genes used for the histograms, scatterplots and boxplots
genes_of_interest <- c("IL24", "ABCA9", "ABCB7")

# Check that each gene exists in the data (should print TRUE TRUE TRUE)
genes_of_interest %in% gene_expression_named$GENE
```

```
## [1] TRUE TRUE TRUE
```

```r
# Reshape the expression data for these three genes
gene_wide <- gene_expression_named %>%
  filter(GENE %in% genes_of_interest) %>%   # keep only rows for our 3 genes
  select(-PROBEID) %>%                      # drop the probe ID column (we use gene names now)
  pivot_longer(-GENE,                       # turn the sample columns into rows:
               names_to = "geoID",          #   column names (sample IDs) go in "geoID"
               values_to = "expression") %>%#   the numbers go in "expression"
  group_by(GENE, geoID) %>%                 # some genes have more than one probe, so
  summarise(expression = mean(expression),  #   average them to get ONE value per gene per person
            .groups = "drop") %>%
  pivot_wider(names_from = GENE,            # now spread back out: one column per gene
              values_from = expression)
# Result: columns geoID, ABCA9, ABCB7, IL24; one row per participant

# Attach the gene columns to the cleaned participant information,
# matching rows by the sample ID (geoID)
```

```r
analysis_df <- participant_clean %>%
  left_join(gene_wide, by = "geoID")

# Remove anyone missing a variable used in the plots
n_before <- nrow(analysis_df)                    # count before removing
analysis_df <- analysis_df %>%
  filter(!is.na(preg_preeclampsia),              # must have an outcome
         !is.na(mat_age_yrs),                    # must have an age
         !is.na(BMI_category))                   # must have a BMI category
n_after <- nrow(analysis_df)                     # count after removing

# Sample sizes for the Methods section
c(initial_samples = ncol(gene_expression) - 1,   # samples in the raw expression file (minus the probe
  matched = n_before,                            # samples with matching participant info
  removed = n_before - n_after,                  # samples removed for missing data
  final = n_after)                               # final sample size
```

```
## initial_samples         matched         removed           final
##             157             157               0             157
```

```r
# Quick look at the first few rows of the gene columns only
head(analysis_df[, c("geoID", genes_of_interest)])
```

```
##        geoID     IL24    ABCA9    ABCB7
## 1 GSM2264363 5.304722 4.712062 7.428219
## 2 GSM2264379 5.345838 4.862162 7.397748
## 3 GSM2264394 5.176851 4.765597 7.652110
## 4 GSM2264418 5.012965 4.674976 7.734432
## 5 GSM2264449 5.302342 4.744986 7.472780
## 6 GSM2264452 5.371454 4.770412 7.314450
```

```r
#FUNCTION THAT MAKES THE 3 PLOTS FOR ANY LIST OF GENES
# Inputs:  df    = a data frame (one row per participant,
#                  one column per gene)
#          genes = a vector of gene names, e.g. c("IL24", "ABCA9")
# Output:  for each gene, a histogram, a scatterplot and a
#          boxplot, shown on screen AND saved as PDFs for LaTeX


# Make a folder called "figures" to save the plots into
# (showWarnings = FALSE means no warning if it already exists)
dir.create("figures", showWarnings = FALSE)

make_gene_plots <- function(df, genes) {

  # One shared theme so all 9 plots look the same.
  # Saving it once means we don't repeat it three times.
  my_theme <- theme_classic(base_size = 14) +
    theme(plot.title = element_text(face = "bold", hjust = 0.5, size = 15), # bold, centered title
          axis.title = element_text(face = "bold", size = 13),              # bold axis titles
          axis.text = element_text(size = 11, color = "black"),             # black tick labels
          legend.title = element_text(face = "bold"))                       # bold legend title
```

```r
# Loop: repeat everything inside { } once for each gene
for (gene in genes) {

  # .data[[gene]] means "the column whose name is stored in gene".
  # On the first loop gene = "IL24", so .data[[gene]] is the IL24 column.
  # This replaces IL24_expression from your Submission 1 code.

  # ---- Plot 1: Histogram ----
  p1 <- ggplot(df, aes(x = .data[[gene]])) +
    geom_histogram(bins = 20,              # number of bars
                   fill = "#8AB9B0",       # bar color
                   color = "white",        # outline between bars
                   linewidth = 0.4,
                   na.rm = TRUE) +         # ignore missing values quietly
    labs(title = paste("Distribution of", gene, "Expression"),  # paste() builds the title from the g
         x = paste(gene, "Expression"),
         y = "Number of Participants") +
    my_theme

  # ---- Plot 2: Scatterplot vs maternal age ----
  p2 <- ggplot(df, aes(x = mat_age_yrs, y = .data[[gene]])) +
    geom_point(size = 2.2, alpha = 0.65,   # alpha = transparency, so overlapping points show
               color = "#C06C84", na.rm = TRUE) +
    geom_smooth(method = "loess",          # flexible trend line
                formula = y ~ x,           # stops R printing a message about the formula
                se = TRUE,                 # shaded 95% confidence band
                color = "#765285", fill = "#D9C2E9", na.rm = TRUE) +
    labs(title = paste(gene, "Expression by Maternal Age"),
         x = "Maternal Age (years)",
         y = paste(gene, "Expression")) +
    my_theme

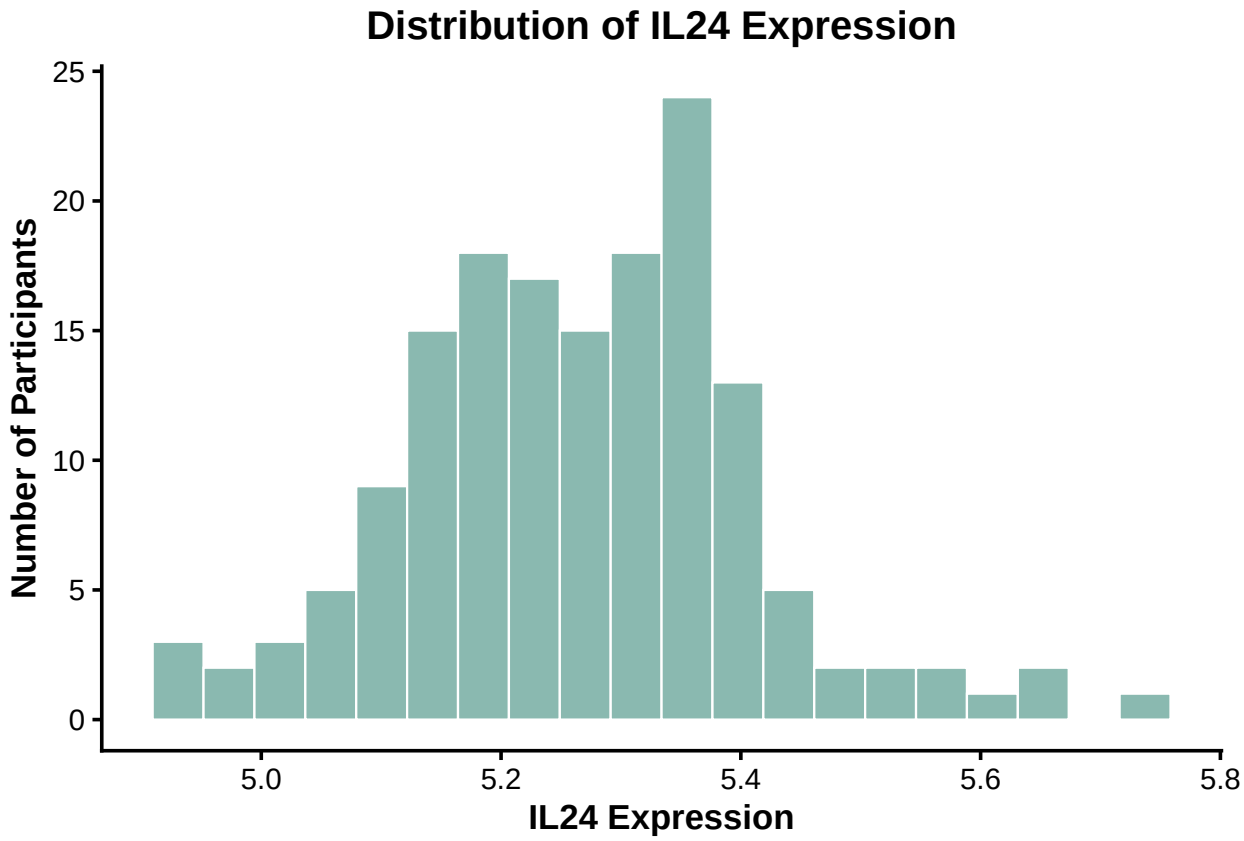
  # ---- Plot 3: Boxplot by outcome and BMI category ----
  p3 <- ggplot(df, aes(x = preg_preeclampsia,     # x-axis: Normal vs Preeclampsia
                       y = .data[[gene]],
                       fill = BMI_category)) +    # separate colored box for each BMI group
    geom_boxplot(position = position_dodge(width = 0.8),  # boxes side by side
                 color = "#51445F", linewidth = 0.4, alpha = 0.75,
                 outlier.shape = NA,      # hide outlier dots (the points below already show them)
                 na.rm = TRUE) +
    geom_point(aes(color = BMI_category),
               position = position_jitterdodge(jitter.width = 0.15,  # spread points out a little
                                               dodge.width = 0.8),   # line them up with their box
               alpha = 0.55, size = 1.5, na.rm = TRUE) +
    scale_fill_manual(values = c("Underweight" = "#B8A1D9",    # box colors
                                 "Normal weight" = "#E8A0BF",
                                 "Overweight" = "#E9B872",
                                 "Obesity" = "#8AB9B0"), drop = FALSE) +
    scale_color_manual(values = c("Underweight" = "#8065A8",   # darker point colors
                                  "Normal weight" = "#B85F85",
                                  "Overweight" = "#B27A32",
                                  "Obesity" = "#4D8278"),
                       guide = "none",    # don't show a second legend for the points
```

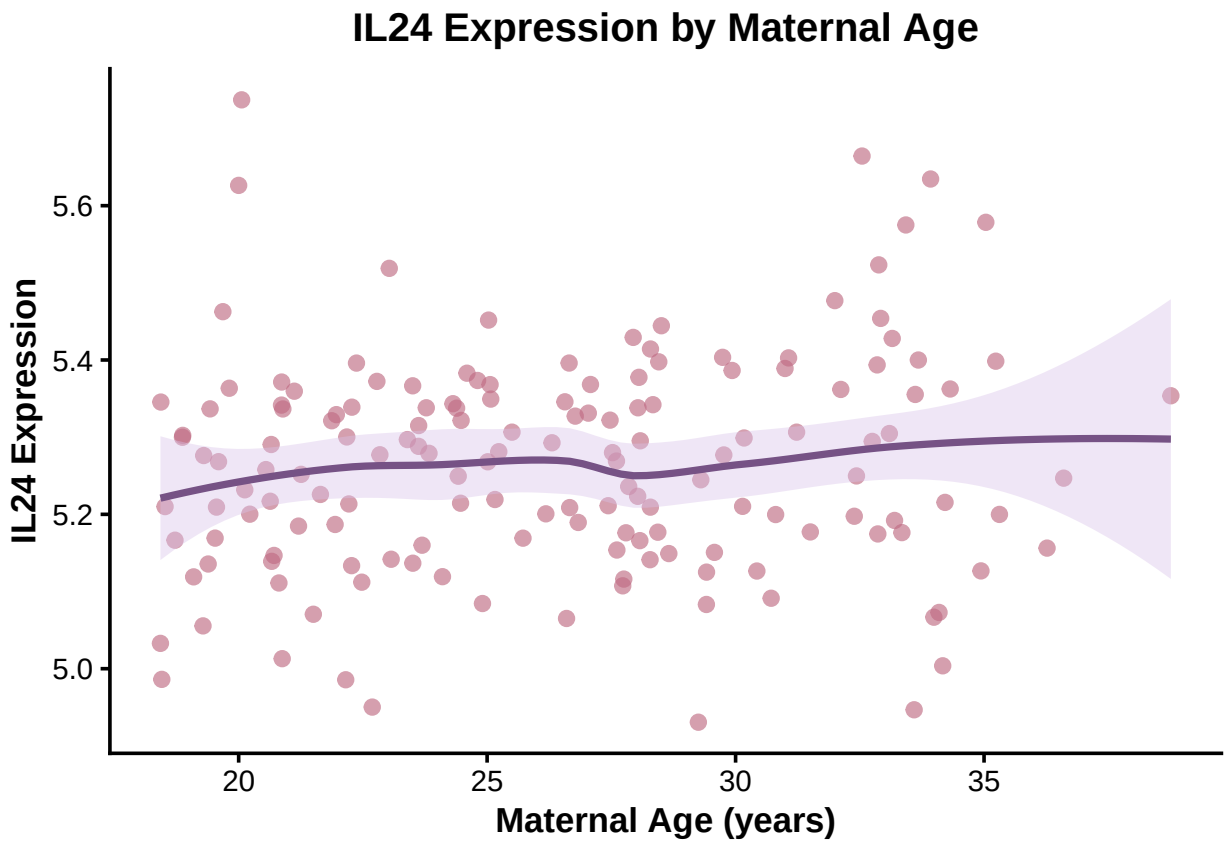
```
                        drop = FALSE) +
      labs(title = paste(gene, "Expression by Outcome and BMI"),
           x = "Pregnancy Outcome",
           y = paste(gene, "Expression"),
           fill = "Maternal BMI\nCategory") +  # \n = line break in the legend title
      my_theme

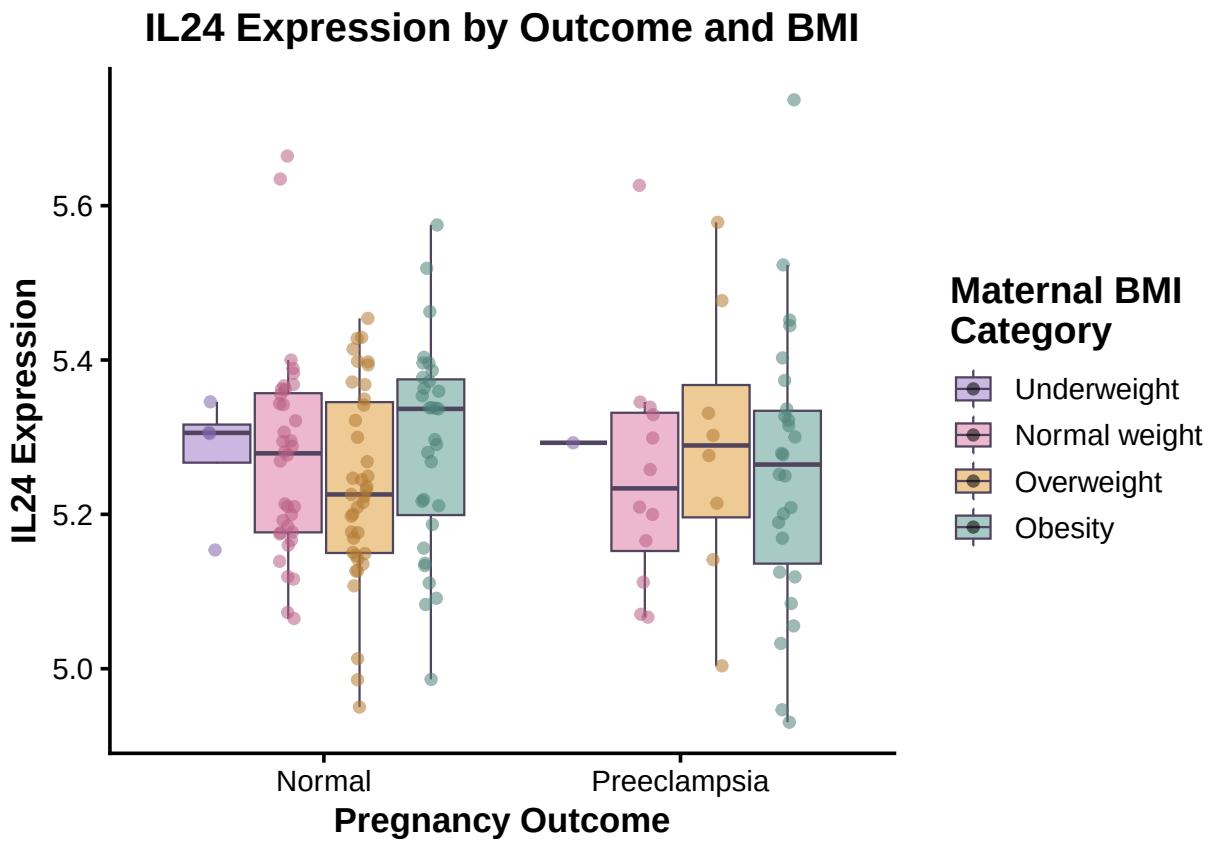
    # Show the three plots (they appear in the knitted PDF)
    print(p1)
    print(p2)
    print(p3)

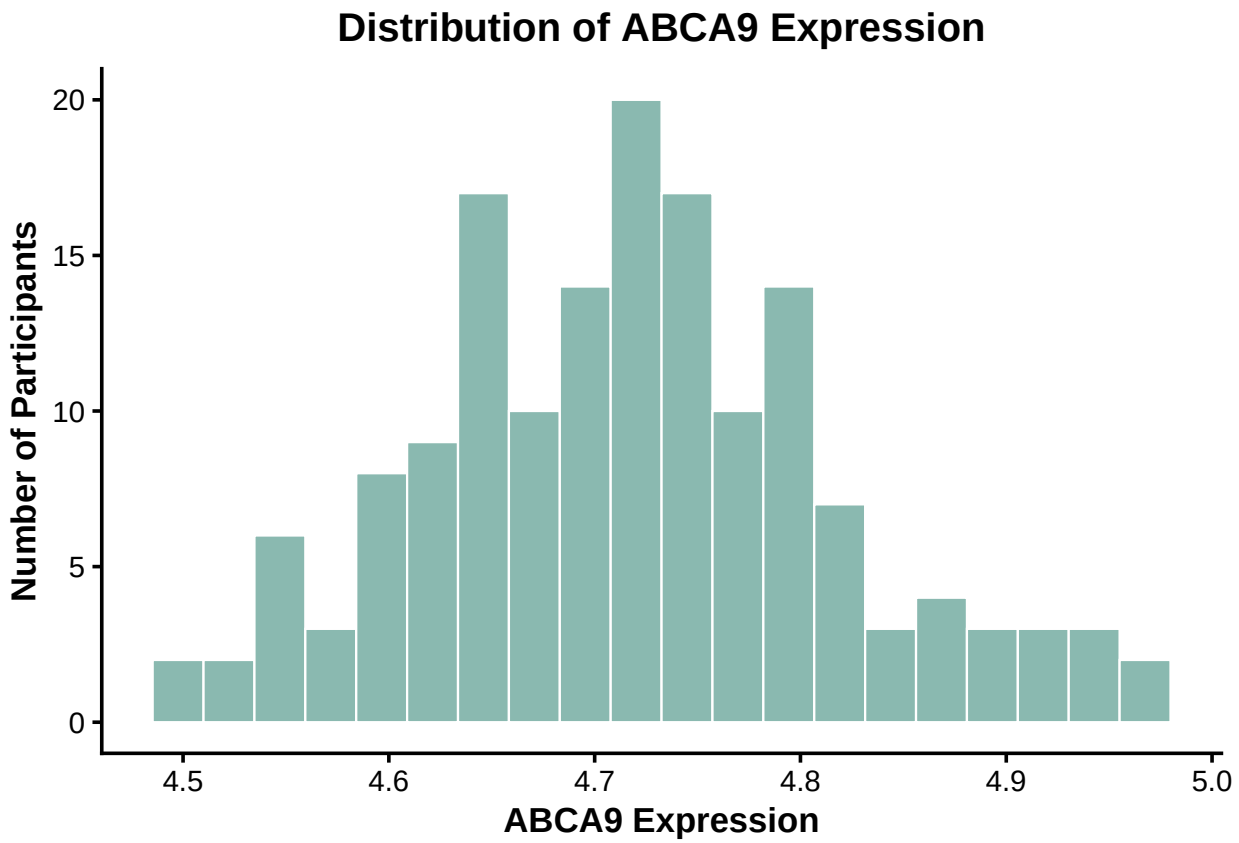
    # Save each plot as a PDF in the figures folder, e.g. figures/IL24_histogram.pdf
    ggsave(paste0("figures/", gene, "_histogram.pdf"), p1, width = 6, height = 4.5)
    ggsave(paste0("figures/", gene, "_scatter.pdf"),   p2, width = 6, height = 4.5)
    ggsave(paste0("figures/", gene, "_boxplot.pdf"),   p3, width = 7.5, height = 4.5)
  }
}

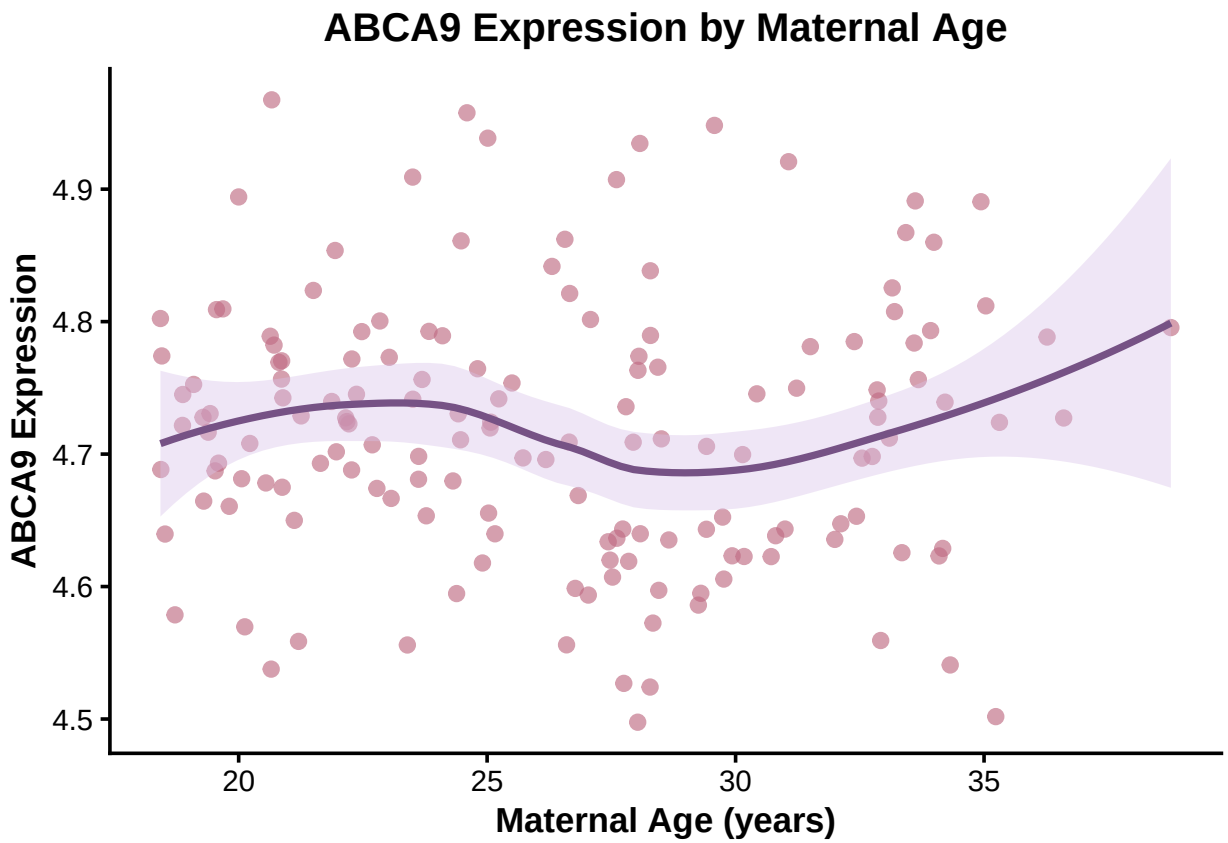
# Use the function: give it the data frame and the 3 genes
make_gene_plots(analysis_df, genes_of_interest)
```

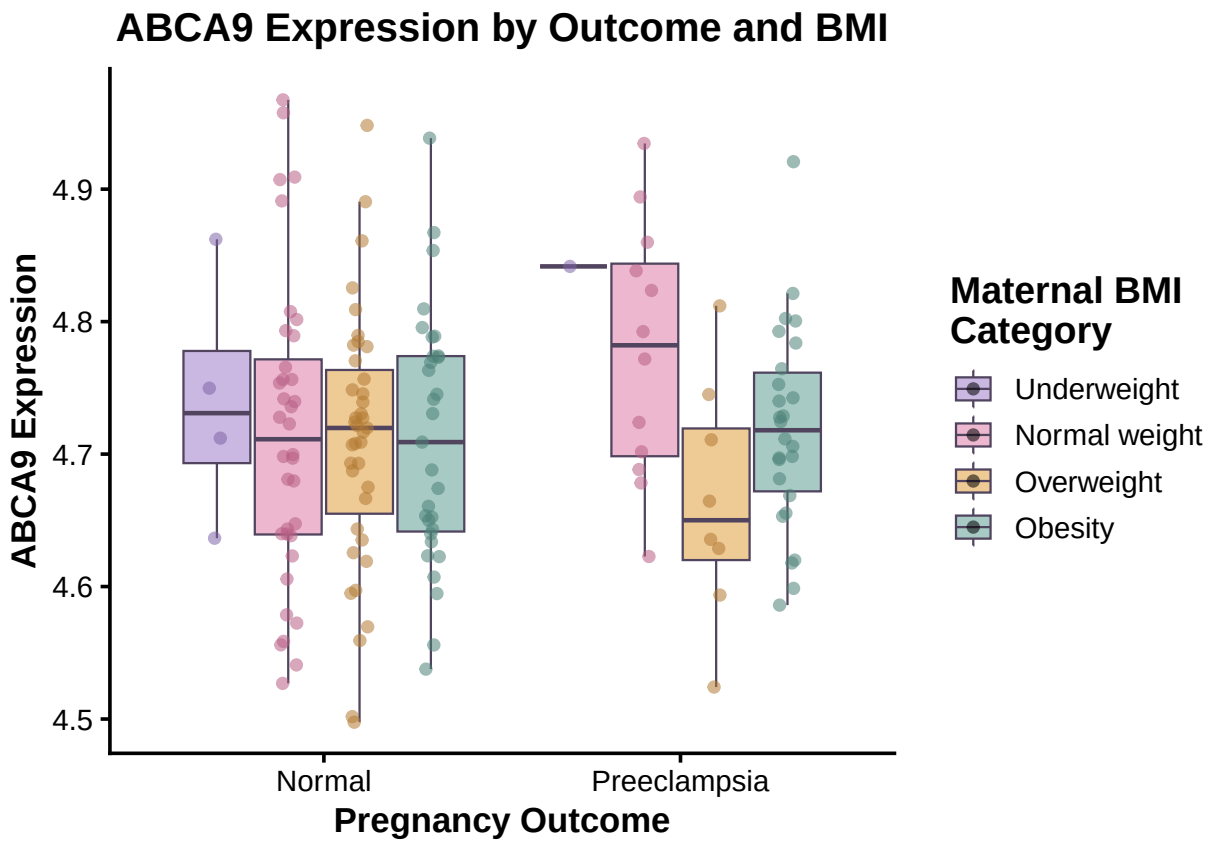


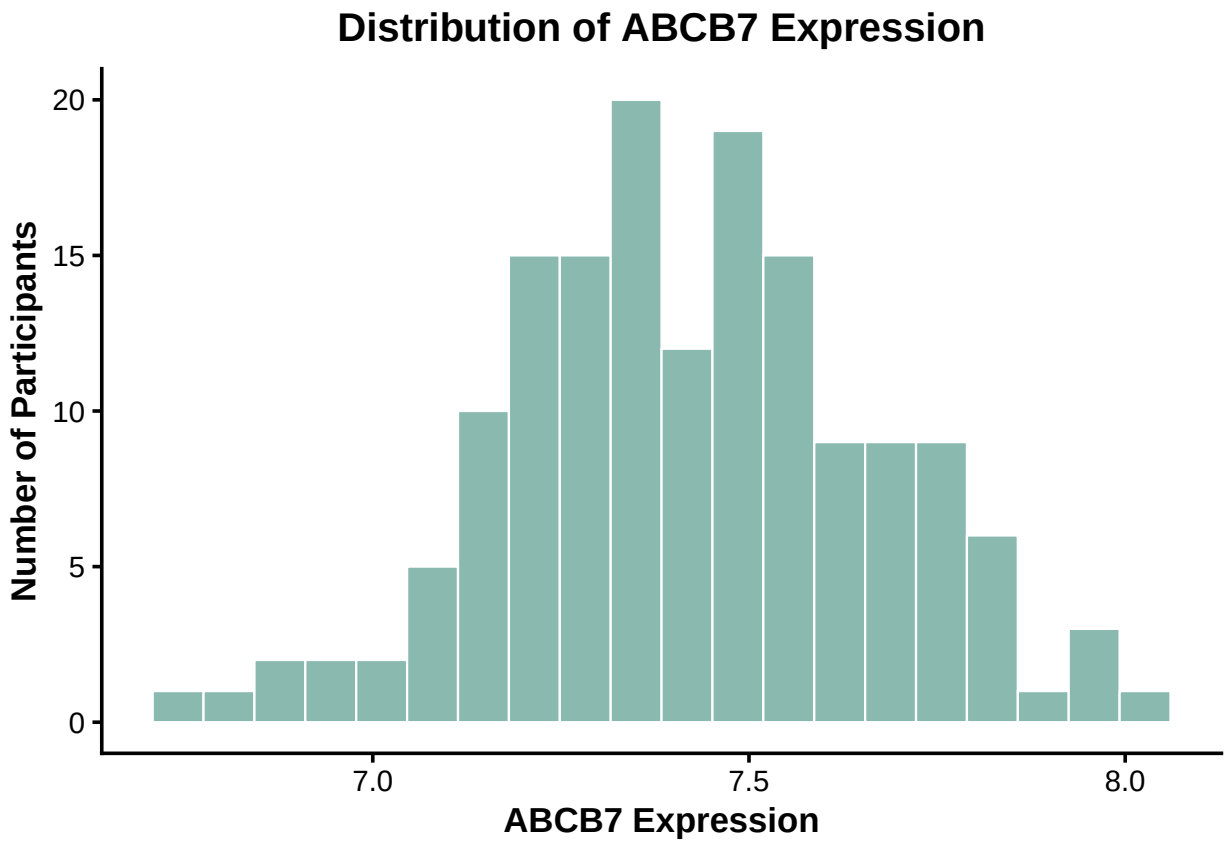

IL24 Expression by Maternal Age
IL24 Expression
5.6
5.4
5.2
5.0
20
25
30
35
Maternal Age (years)

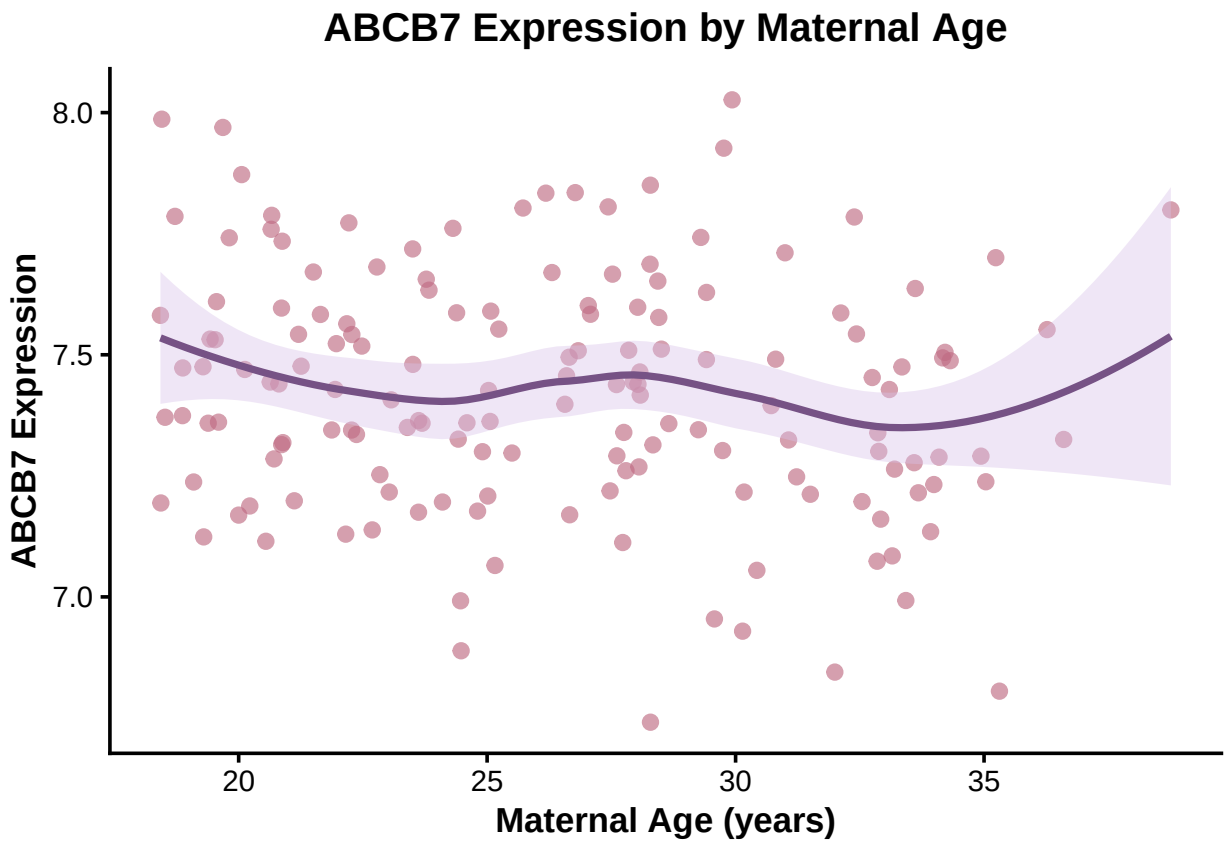

IL24 Expression by Outcome and BMI
IL24 Expression
5.6
5.4
5.2
5.0
Normal
Preeclampsia
Pregnancy Outcome
Maternal BMI Category
Underweight
Normal weight
Overweight
Obesity

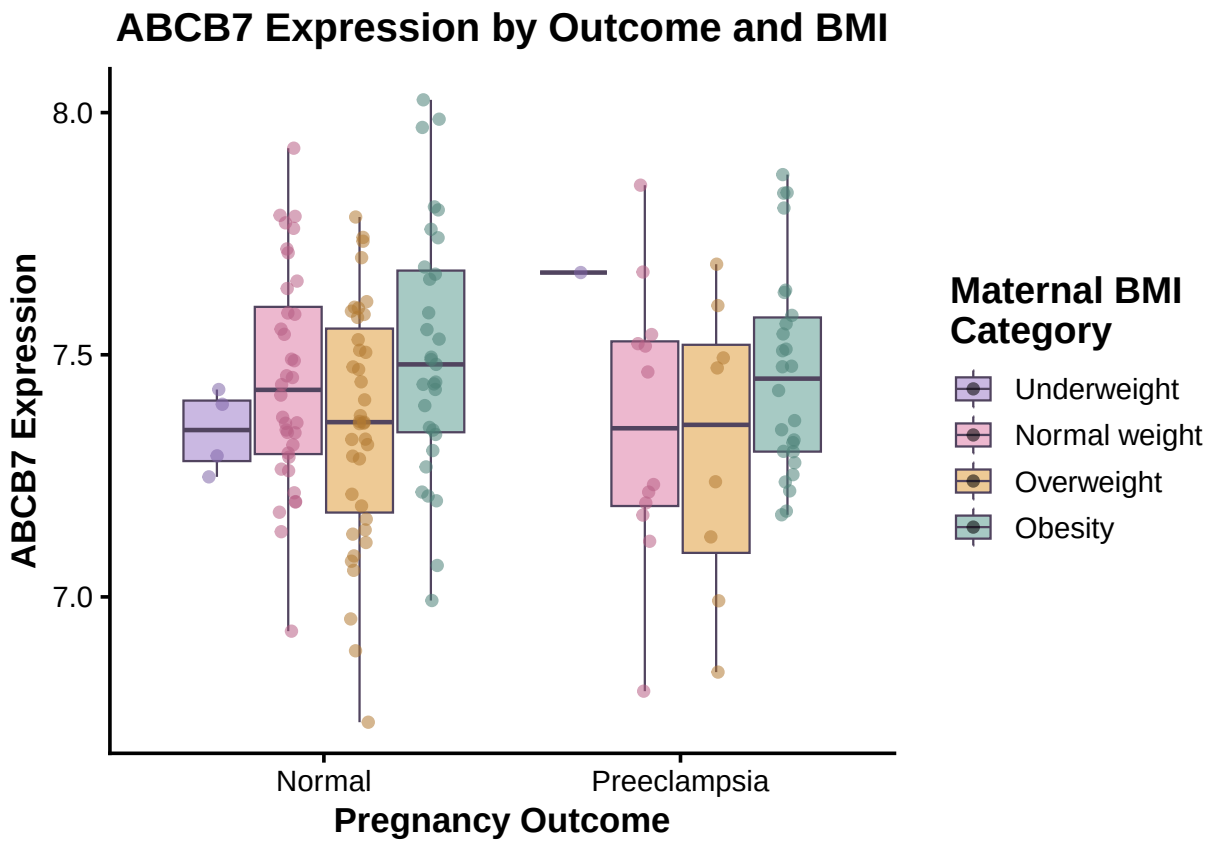

Distribution of ABCA9 Expression
Number of Participants
20
15
10
5
0
4.5
4.6
4.7
4.8
4.9
5.0
ABCA9 Expression


ABCA9 Expression by Maternal Age
ABCA9 Expression
4.9
4.8
4.7
4.6
4.5
20
25
30
35
Maternal Age (years)


ABCA9 Expression by Outcome and BMI
ABCA9 Expression
4.9
4.8
4.7
4.6
4.5
Normal
Preeclampsia
Pregnancy Outcome
Maternal BMI Category
Underweight
Normal weight
Overweight
Obesity


Distribution of ABCB7 Expression
Number of Participants
20
15
10
5
0
7.0
7.5
8.0
ABCB7 Expression


ABCB7 Expression by Maternal Age
ABCB7 Expression
8.0
7.5
7.0
20
25
30
35
Maternal Age (years)




```
# SIMPLE STATISTICAL TESTS
# For each gene we ask three questions:
#  1. Is expression related to maternal AGE?        -> Spearman correlation
#  2. Is expression different between Normal and
#     Preeclampsia?                                  -> Wilcoxon rank-sum test
#  3. Is expression different across the 4 BMI
#     categories?                                    -> Kruskal-Wallis test
# These tests don't assume the data are normally distributed,
# which makes them a safe choice.
# A p-value below 0.05 is usually read as "associated".
# ------------------------------------------------------------

#  three tests for one gene and return the results as one row
run_tests <- function(gene) {
  expr <- analysis_df[[gene]]   # the expression values for this gene

  # 1. Correlation with age: rho runs from -1 to 1
  #    (positive = goes up with age, negative = goes down, near 0 = no relationship)
  age_test <- cor.test(expr, analysis_df$mat_age_yrs,
                       method = "spearman", exact = FALSE)

  # 2. Normal vs Preeclampsia
  outcome_test <- wilcox.test(expr ~ analysis_df$preg_preeclampsia)

  # 3. Across the four BMI categories
  bmi_test <- kruskal.test(expr ~ analysis_df$BMI_category)
```

```r
  # Put the results in one row, rounded to 3 decimal places
  data.frame(gene      = gene,
             rho_age   = round(age_test$estimate, 3),
             p_age     = round(age_test$p.value, 3),
             p_outcome = round(outcome_test$p.value, 3),
             p_BMI     = round(bmi_test$p.value, 3),
             row.names = NULL)
}

# Run the tests for all 3 genes and stack the rows into one table
test_results <- do.call(rbind, lapply(genes_of_interest, run_tests))
test_results
```

```
##    gene rho_age p_age p_outcome p_BMI
## 1  IL24   0.118 0.140     0.601 0.782
## 2 ABCA9  -0.042 0.600     0.429 0.489
## 3 ABCB7  -0.152 0.057     0.814 0.149
```

```r
# SUMMARY STATISTICS TABLE (for LaTeX)
# Rows:    3 continuous variables (shown as mean (SD))
#          + BMI category (shown as n (%))
# Columns: Overall, Normal, Preeclampsia


# Helper 1: format a continuous variable as "mean (SD)", e.g. "26.4 (5.1)"
# sprintf() builds text from numbers; "%.1f" means 1 decimal place
mean_sd <- function(x) {
  sprintf("%.1f (%.1f)", mean(x, na.rm = TRUE), sd(x, na.rm = TRUE))
}

# Helper 2: format one category as "n (%)", e.g. "57 (36.3)"
# sum(x == lev) counts people in that category;
# mean(x == lev) gives the proportion, times 100 for a percent
n_pct <- function(x, lev) {
  sprintf("%d (%.1f)", sum(x == lev, na.rm = TRUE),
          100 * mean(x == lev, na.rm = TRUE))
}

# Helper 3: build one whole COLUMN of the table for a group of people
make_column <- function(d) {
  c(mean_sd(d$mat_age_yrs),               # row 1: age
    mean_sd(d$mat_vitD),                  # row 2: vitamin D
    mean_sd(d$delivery_gestAge_wks),      # row 3: gestational age at delivery
    "",                                   # row 4: blank (BMI heading row)
    sapply(levels(d$BMI_category),        # rows 5-8: one row per BMI category
           function(l) n_pct(d$BMI_category, l)))
}

# First column: the row labels
# "\\%" prints a % sign in LaTeX (a plain % would start a LaTeX comment)
# "\\quad" indents the BMI categories under their heading
summary_table <- data.frame(
  Characteristic = c("Maternal age (years), mean (SD)",
```

```
                      "Vitamin D (ng/mL), mean (SD)",
                      "Gestational age at delivery (weeks), mean (SD)",
                      "Maternal BMI category, n (\\%)",
                      paste0("\\quad ", levels(analysis_df$BMI_category))))

# Add the Overall column (everyone), with the sample size in the header
summary_table[[paste0("Overall (n = ", nrow(analysis_df), ")")]] <-
  make_column(analysis_df)

# Add one column per outcome group (Normal, then Preeclampsia)
for (g in levels(analysis_df$preg_preeclampsia)) {
  d <- filter(analysis_df, preg_preeclampsia == g)   # people in this group only
  summary_table[[paste0(g, " (n = ", nrow(d), ")")]] <- make_column(d)
}

# Turn the table into LaTeX code
# booktabs = TRUE gives clean publication-style lines
# escape = FALSE keeps our \\% and \\quad working
# align = "lccc": first column left-aligned, the other 3 centered
latex_table <- knitr::kable(summary_table, format = "latex",
                            booktabs = TRUE, escape = FALSE, align = "lccc")

# Save the LaTeX code to a file (uploaded to Overleaf later)
writeLines(latex_table, "table1.tex")

# Also show the table here so you can check the numbers
knitr::kable(summary_table)
```

Characteristic	Overall (n = 157)	Normal (n = 110)	Preeclampsia (n = 47)
Maternal age (years), mean (SD)	26.4 (5.0)	26.6 (5.1)	25.8 (4.9)
Vitamin D (ng/mL), mean (SD)	23.0 (13.1)	24.4 (14.5)	19.7 (8.3)
Gestational age at delivery (weeks), mean (SD)	38.8 (2.2)	39.3 (1.0)	37.8 (3.5)
Maternal BMI category, n (%)			
Underweight	5 (3.2)	4 (3.6)	1 (2.1)
Normal weight	48 (30.6)	36 (32.7)	12 (25.5)
Overweight	47 (29.9)	39 (35.5)	8 (17.0)
Obesity	57 (36.3)	31 (28.2)	26 (55.3)

```
#HEATMAP

# STEP 8a: CHOOSE AND CHECK THE 10 HEATMAP GENES
# My 3 genes + 7 other ABC transporter genes (the same gene
# family as ABCA9 and ABCB7)

heatmap_genes <- c("IL24", "ABCA9", "ABCB7",                  # my 3 genes
                   "ANKRD60", "ATG101", "CATSPERT", "ADAR",        # 7 more ABC transporters
                   "AKR1C8",  "BHMT", "CCDC87")

# Which of these genes are NOT in the data?
```

```r
# character(0) means none are missing, so all 10 were found
setdiff(heatmap_genes, gene_expression_named$GENE)
```

```
## character(0)
```

```r
# STEP 8b: HEATMAP OF 10 GENES
# Each column = one participant, each row = one gene.
# Color = how high or low a person's expression is compared with
# everyone else for that gene. Similar genes and similar people
# are grouped together (clustering). Colored bars on top show
# each person's pregnancy outcome and BMI category.


# 1. a matrix of the 10 genes (rows) x 157 participants (columns)
heat_df <- gene_expression_named %>%
  filter(GENE %in% heatmap_genes) %>%      # keep only the 10 genes
  select(-PROBEID) %>%                     # drop the probe ID column
  group_by(GENE) %>%                       # if a gene has several probes...
  summarise(across(everything(), mean))    # ...average them into one row

heat_mat <- as.matrix(heat_df[, -1])       # numbers only (drop the GENE column)
rownames(heat_mat) <- heat_df$GENE         # use gene names as row names
heat_mat <- heat_mat[, analysis_df$geoID]  # same participants, same order as analysis_df

dim(heat_mat)                              # should print 10 157
```

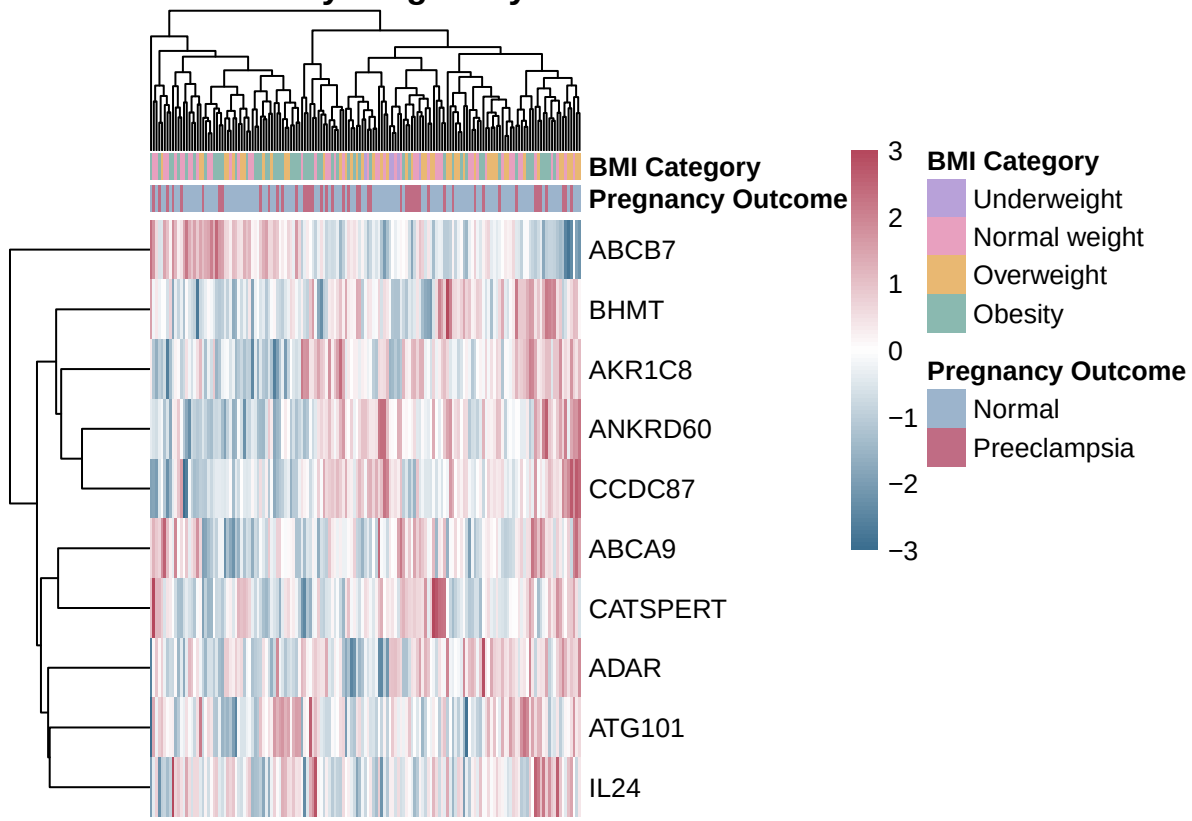
```
## [1]  10 157
```

```r
# 2. Tracking bars: one row per participant, one column per categorical variable
#    row.names must be the geoIDs so pheatmap can match bars to columns
annotation <- data.frame(
  "Pregnancy Outcome" = analysis_df$preg_preeclampsia,
  "BMI Category"      = analysis_df$BMI_category,
  row.names   = analysis_df$geoID,
  check.names = FALSE)                     # keeps the spaces in the column names

# 3. Colors for the tracking bars (the BMI colors match the boxplots)
annotation_colors <- list(
  "Pregnancy Outcome" = c("Normal" = "#9CB4CC", "Preeclampsia" = "#C06C84"),
  "BMI Category"      = c("Underweight" = "#B8A1D9", "Normal weight" = "#E8A0BF",
                          "Overweight" = "#E9B872", "Obesity" = "#8AB9B0"))

# 4. Draw the heatmap
heatmap_plot <- pheatmap(
  heat_mat,
  scale = "row",                           # z-score each gene so all genes share one color scale
  clustering_distance_rows = "euclidean", # how similarity between genes is measured
  clustering_distance_cols = "euclidean", # how similarity between participants is measured
  clustering_method = "complete",          # how clusters are joined together
  annotation_col = annotation,             # the tracking bars on top
  annotation_colors = annotation_colors,
  show_colnames = FALSE,                   # hide 157 sample IDs (unreadable anyway)
```

```
  color = colorRampPalette(c("#3B6E8F", "white", "#B5485D"))(100), # blue = low, red = high
  breaks = seq(-3, 3, length.out = 101),  # cap colors at +/- 3 so outliers don't wash out the rest
  border_color = NA,                      # no grid lines between cells
  main = "Expression of 10 Genes by Pregnancy Outcome and BMI")
```




```
# 5. Save the heatmap as a PDF in the figures folder
pdf("figures/heatmap.pdf", width = 8, height = 5.5)  # open a PDF file
grid::grid.draw(heatmap_plot$gtable)                 # draw the heatmap into it
dev.off()                                            # close and save the file
```

```
## pdf
##   2
```

```
#additional plot

# DUMBBELL PLOT
# For each of the 10 heatmap genes: the mean expression in Normal
# vs Preeclampsia pregnancies, joined by a bar. Genes are z-scored
# so all 10 share the same scale, and they are sorted by how big
# the difference between groups is.
# ------------------------------------------------------------------

#  Z-score each gene (mean 0, SD 1 across all participants)
#    scale() works on columns, so we flip (t) the matrix, scale, then flip back
```

```r
z_mat <- t(scale(t(heat_mat)))

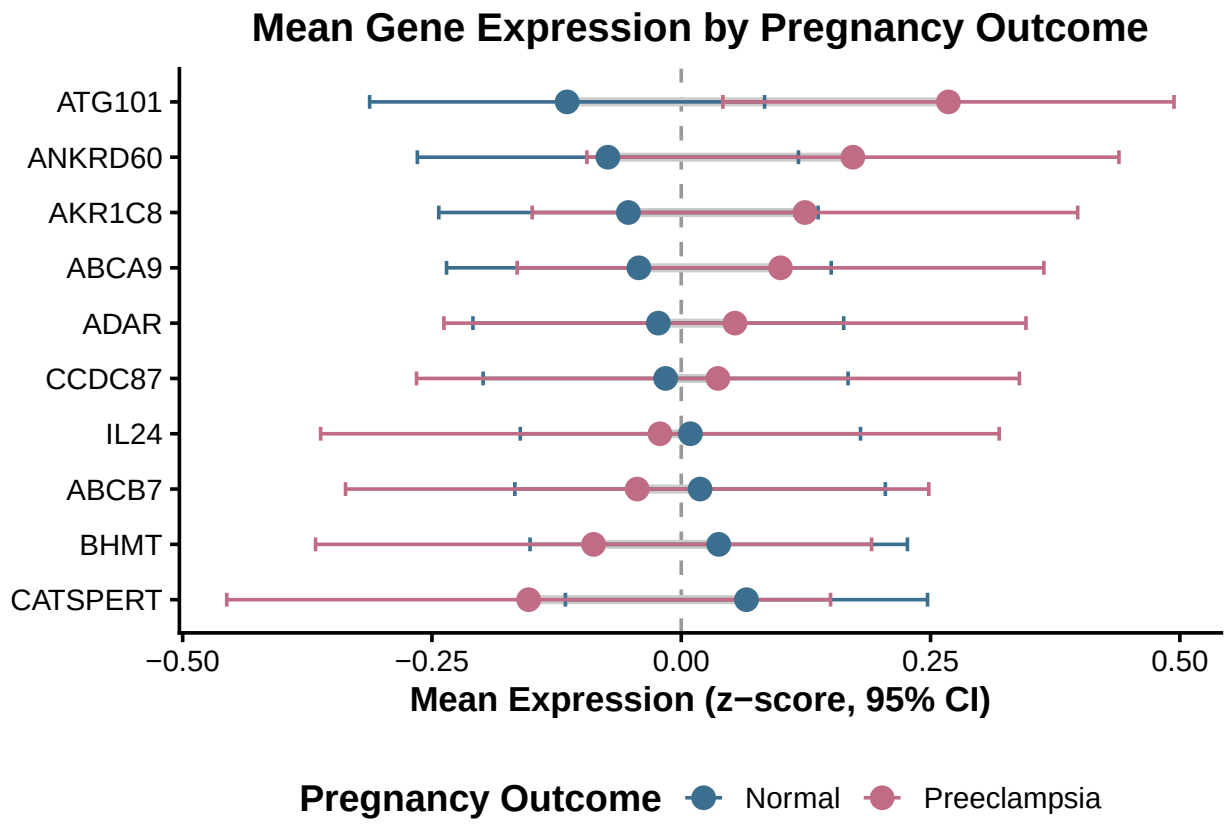
# 2. Mean and 95% confidence interval for each gene in each group
group_means <- as.data.frame(t(z_mat)) %>%            # rows = participants, columns = genes
  mutate(group = analysis_df$preg_preeclampsia) %>%   # add each person's outcome
  pivot_longer(-group, names_to = "gene",             # long format: one row per person per gene
               values_to = "z") %>%
  group_by(gene, group) %>%                           # for each gene in each group...
  summarise(mean_z = mean(z),                         # ...the mean
            se = sd(z) / sqrt(n()),                   # ...the standard error
            .groups = "drop") %>%
  mutate(lower = mean_z - 1.96 * se,                  # lower end of the 95% CI
         upper = mean_z + 1.96 * se)                  # upper end of the 95% CI

# One row per gene with both group means side by side (used for the grey bar)
gene_gaps <- group_means %>%
  select(gene, group, mean_z) %>%
  pivot_wider(names_from = group, values_from = mean_z) %>%  # columns: gene, Normal, Preeclampsia
  mutate(difference = Preeclampsia - Normal) %>%             # size of the gap between groups
  arrange(difference)                                        # sort genes by the gap

#  Keep the genes in that sorted order on the y-axis
group_means$gene <- factor(group_means$gene, levels = gene_gaps$gene)
gene_gaps$gene   <- factor(gene_gaps$gene,   levels = gene_gaps$gene)

# Draw the plot (layers are added from back to front)
plot_dumbbell <- ggplot() +
  geom_vline(xintercept = 0, linetype = "dashed",     # dashed line at 0 = overall average
             color = "grey60") +
  geom_segment(data = gene_gaps,                      # grey bar joining the two group means
               aes(x = Normal, xend = Preeclampsia, y = gene, yend = gene),
               linewidth = 1.6, color = "grey80") +
  geom_errorbar(data = group_means,                   # 95% CI whiskers
                aes(xmin = lower, xmax = upper, y = gene, color = group),
                width = 0.25, orientation = "y") +
  geom_point(data = group_means,                      # the mean dots
             aes(x = mean_z, y = gene, color = group), size = 3.5) +
  scale_color_manual(values = c("Normal" = "#3B6E8F",
                                "Preeclampsia" = "#C06C84")) +
  labs(title = "Mean Gene Expression by Pregnancy Outcome",
       x = "Mean Expression (z-score, 95% CI)",
       y = NULL,                                      # gene names are self-explanatory
       color = "Pregnancy Outcome") +
  theme_classic(base_size = 14) +
  theme(plot.title = element_text(face = "bold", hjust = 0.5, size = 15),
        axis.title = element_text(face = "bold", size = 13),
        axis.text = element_text(size = 11, color = "black"),
        legend.title = element_text(face = "bold"),
        legend.position = "bottom")

#  Show the plot and save it for LaTeX
print(plot_dumbbell)
```




```
ggsave("figures/dumbbell.pdf", plot_dumbbell, width = 7, height = 5)

#  The differences, sorted (useful for writing the Results)
gene_gaps
```

```
## # A tibble: 10 x 4
##    gene       Normal Preeclampsia difference
##    <fct>       <dbl>        <dbl>      <dbl>
##  1 CATSPERT  0.0654       -0.153     -0.218
##  2 BHMT      0.0376       -0.0879    -0.126
##  3 ABCB7     0.0189       -0.0442    -0.0631
##  4 IL24      0.00915      -0.0214    -0.0306
##  5 CCDC87   -0.0157        0.0368     0.0525
##  6 ADAR     -0.0230        0.0538     0.0768
##  7 ABCA9    -0.0425        0.0995     0.142
##  8 AKR1C8   -0.0530        0.124      0.177
##  9 ANKRD60  -0.0736        0.172      0.246
## 10 ATG101   -0.114         0.268      0.382
```

```
#  SOFTWARE VERSIONS AND CITATIONS
# The Methods must state which versions of R and each package
# were used, and the References must cite them.


# Version of R
R.version.string
```

```
## [1] "R version 4.6.1 (2026-06-24)"
```

```r
# Version of each package used
packages_used <- c("ggplot2", "dplyr", "tidyr", "pheatmap", "knitr")
sapply(packages_used, function(p) as.character(packageVersion(p)))
```

```
##  ggplot2    dplyr    tidyr pheatmap    knitr
##  "4.0.3"  "1.2.1"  "1.3.2" "1.0.13"   "1.51"
```

```r
# Write a BibTeX citation for R ("base") and each package into a
# file called packages.bib (uploaded to Overleaf later)
knitr::write_bib(c("base", packages_used), file = "packages.bib")
```

Resources: I used Claude (Anthropic), an AI assistant, for help with structuring and debugging code for the plotting function, summary statistics table, heatmap and dumbbell plot.